Studio Nx

JS 25 — initPasswordForm()

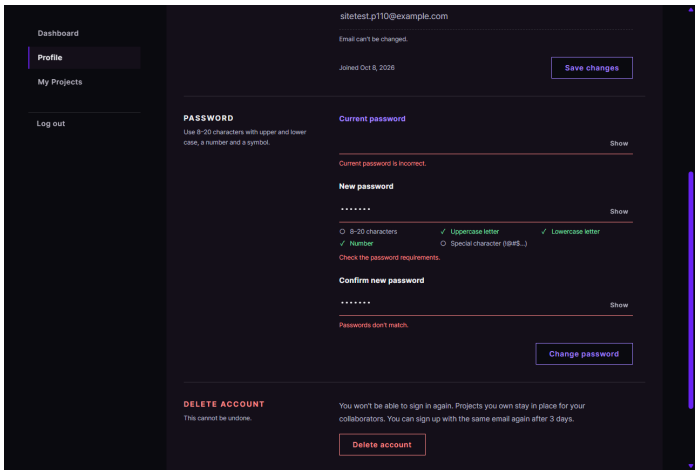
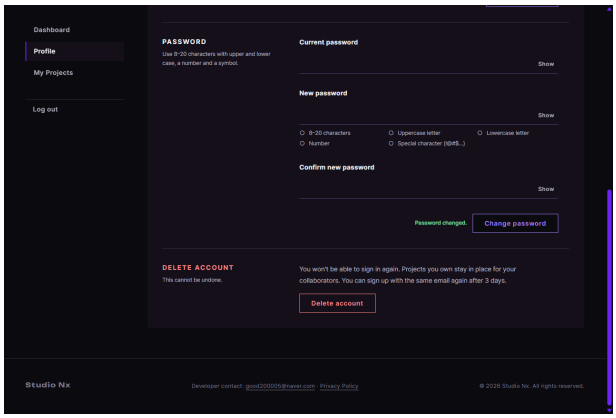
2026-10-07

JS 25 — `initPasswordForm()` (+ `core/password.js`)

항목	내용
파일	`studionx/frontend/static/js/pages/userinfo.js` · 새 `static/js/core/password.js` · `pages/join.js` (import 한 줄) · `userInfo.html` (문구 자리)
작업	P1-10 회원 정보 화면 JS — 비밀번호 변경 · PRD 01 FR-107
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	24 `initProfileForm()` · 26 `initWithdraw()`

1. 하는 일

회원 정보 화면의 **Password** 칸 세 개(지금 · 새 · 확인)를 가입 화면과 같은 방식으로 검사하고, **Change password** 로 바꾼다.

고칠 곳 표시	바뀜
	

한 일	화면
새 비밀번호 `Studio1` 입력	조건 목록 대 · 소 · 숫자 ✓ (입력 중엔 빨간 글 없음) → 칸을 벗어나면 "Check the password requirements."
새 비밀번호에 한글	바로 "Use only English letters, numbers and symbols."
확인 칸이 다름	같은 길이까지 쳤거나 칸을 벗어나면 "Passwords don't match." · 같아지면 바로 지움
빈 채로 Change password	세 칸에 문구, 커서는 지금 비밀번호 칸, **보내지 않음**
지금 비밀번호 틀림 (서버 400)	"Current password is incorrect." · 그 칸 비우고 커서 — **로그아웃되지 않음**
새 = 지금 (서버 422)	새 비밀번호 칸 "Use a different password." (P1-09 사용자 결정)
바뀜 (204)	세 칸 비움 · 조건 ○ 로 · 버튼 옆 초록 "Password changed." (5초)

2. 언제 쓰나

userInfo.html 이 열릴 때 한 번. 서버는 백엔드 47 `PUT /users/me/password`.

규칙 파일을 나눔 — `core/password.js` (템플릿 권장)

전	지금
`PASSWORD_RULES` · `PASSWORD_ALLOWED` 가 `pages/join.js` 안에	`core/password.js` 로 옮기고, `join.js` 와 `userinfo.js` 가 함께 import

이유: `userinfo.js` 가 `join.js` 를 import 하면 **join 페이지의 코드(맨 아래 init 줄들)까지 실행**된다. 규칙만 따로 두면 두 화면이 같은 규칙을 쓴다. `join.js` 는 import 한 줄만 바뀌었고, 가입 화면 시험(P1-07 16 · 17)을 다시 돌려 모두 통과했다. P1-07 16 문서의 "규칙 위치" 줄도 고쳤다.

HTML 에 더한 것: Change password 버튼 옆 `<span id="password-saved" role="status">` (24 의 Saved 와 같은 모양 · 같은 CSS).

3. 코드

import (파일 맨 위)

```
import { api } from '../core/api.js';
import { PASSWORD_RULES, PASSWORD_ALLOWED } from '../core/password.js';
```

`core/password.js` 줄을 이번에 추가했다.

`core/password.js` (새 파일 — `join.js` 에 있던 것 그대로)

```
export const PASSWORD_RULES = {
  length: (pw) => pw.length >= 8 && pw.length <= 20,
  upper: (pw) => /[A-Z]/.test(pw),
  lower: (pw) => /[a-z]/.test(pw),
  number: (pw) => /[0-9]/.test(pw),
  special: (pw) => /[!-/:-@[-`{-~]/.test(pw),
};
export const PASSWORD_ALLOWED = /^[!-~]*$/;
```

함수 — 준비 · 입력할 때

```
export function initPasswordForm() {
  const form = document.getElementById('password-form');
  const current = document.getElementById('pw-current');
  const password = document.getElementById('pw-new');
  const confirm = document.getElementById('pw-confirm');
  const currentField = document.getElementById('pw-current-field');
  const passwordField = document.getElementById('pw-new-field');
  const confirmField = document.getElementById('pw-confirm-field');
  const items = document.querySelectorAll('#pw-new-rules li[data-rule]');
  const error = document.getElementById('password-error');
  const saved = document.getElementById('password-saved');
  const submit = form ? form.querySelector('button[type="submit"]') : null;
  if (!form || !current || !password || !confirm) return;
  let savedTimer = null;

  const allMet = () => {
    if (!PASSWORD_ALLOWED.test(password.value)) return false;
```

```
  for (const name of Object.keys(PASSWORD_RULES)) {
    if (!PASSWORD_RULES[name](password.value)) return false;
  }
  return true;
};

const showRules = () => {
  for (const item of items) {
    const rule = PASSWORD_RULES[item.dataset.rule];
    const met = rule ? rule(password.value) : false;
    item.classList.toggle('is-met', met);
    const state = item.querySelector('[data-rule-state]');
    if (state) state.textContent = met ? ', met' : ', not met';
  }
};

const checkConfirm = (force) => {
  if (confirm.value === '' || confirm.value === password.value) {
    setFieldState(confirmField, null);
  } else if (force || confirm.value.length >= password.value.length) {
    setFieldState(confirmField, 'error', "Passwords don't match.");
  }
};

const clearSaved = () => {
  if (saved) saved.textContent = '';
};

current.addEventListener('input', () => {
  setFieldState(currentField, null);
  clearSaved();
});
password.addEventListener('input', () => {
  showRules();
  clearSaved();
  if (!PASSWORD_ALLOWED.test(password.value)) {
    setFieldState(passwordField, 'error', 'Use only English letters, numbers and symbols.');
  } else {
    setFieldState(passwordField, null);
  }
  if (confirm.value !== '') checkConfirm(true);
});
password.addEventListener('blur', () => {
  if (password.value !== '' && PASSWORD_ALLOWED.test(password.value) && !allMet()) {
    setFieldState(passwordField, 'error', 'Check the password requirements.');
  }
});
confirm.addEventListener('input', () => {
  clearSaved();
  checkConfirm(false);
});
confirm.addEventListener('blur', () => checkConfirm(true));
```

함수 — 보내기

```
  form.addEventListener('submit', async (event) => {
    event.preventDefault();
    if (error) error.hidden = true;
    const problems = [];
    if (current.value === '') {
      setFieldState(currentField, 'error', 'Enter your current password.');
      problems.push(current);
    }
    if (!allMet()) {
      setFieldState(passwordField, 'error', 'Check the password requirements.');
      problems.push(password);
    }
    if (confirm.value !== password.value) {
      setFieldState(confirmField, 'error', "Passwords don't match.");
      problems.push(confirm);
    }
    if (problems.length > 0) {
      problems[0].focus();
      return;
    }

    if (submit) submit.disabled = true;
    try {
      await api.put('/users/me/password', { current_password: current.value, new_password: password.valu
e });
      current.value = '';
      password.value = '';
      confirm.value = '';
      setFieldState(currentField, null);
      setFieldState(passwordField, null);
      setFieldState(confirmField, null);
      showRules();
      if (saved) {
        saved.textContent = 'Password changed.';
        clearTimeout(savedTimer);
        savedTimer = setTimeout(clearSaved, 5000);
      }
    } catch (err) {
      if (err.code === 'WRONG_PASSWORD') {
        setFieldState(currentField, 'error', 'Current password is incorrect.');
        current.value = '';
        current.focus();
      } else if (err.status === 422 && err.details && err.details.new_password) {
        setFieldState(passwordField, 'error', err.message);
        password.focus();
      } else if (error) {
        error.textContent = err.message || 'Something went wrong. Please try again.';
        error.hidden = false;
      }
    } finally {
      if (submit) submit.disabled = false;
    }
  });
}
```

최신 문법으로 쓰면 (공부용 — 바뀌는 부분만)

```
const allMet = () => PASSWORD_ALLOWED.test(password.value)
  && Object.values(PASSWORD_RULES).every((rule) => rule(password.value));
…
const met = PASSWORD_RULES[item.dataset.rule]?.(password.value) ?? false;
item.querySelector('[data-rule-state]')?.replaceChildren(met ? ', met' : ', not met');
…
} else if (err.status === 422 && err.details?.new_password) {
```

최신 문법	실제 코드	뜻
`Object.values(…).every(fn)`	`for … of Object.keys(…)` 로 돌다가 하나라도 틀리면 `return false`	모두 참인지
`rule?.(value) ?? false`	`rule ? rule(value) : false`	함수가 있을 때만 부르고, 없으면 `false`
`el?.replaceChildren(글)`	`if (state) state.textContent = 글`	있을 때만 글자 바꾸기
`err.details?.new_password`	`err.details && err.details.new_password`	`details` 가 있을 때만

문법 풀이

import 문법	뜻
`import { PASSWORD_RULES, PASSWORD_ALLOWED } from '../core/password.js'`	새 공용 파일의 이름 있는 내보내기 둘

식	뜻
`const problems = []` · `problems.push(input)` · `problems[0].focus()`	틀린 칸을 화면 위쪽부터 모아 첫 칸에 커서 (가입 17 과 같음)
`err.code === 'WRONG_PASSWORD'`	서버(P1-09 44)가 400 과 함께 주는 코드
`err.details.new_password`	서버 422 의 칸 이름 — 규칙 목록이거나 `'same_as_current'`
`setTimeout(clearSaved, 5000)`	5초 뒤 문구 지우기 (함수를 그대로 넘김)

4. 흐름

입력할 때: 지금 칸 → 오류 지움 / 새 칸 → 조건 목록 · 한글 검사 · 확인 칸 다시 비교 / 확인 칸 → 다 쳤으면 비교. 어느 칸이든 입력하면 "Password changed." 를 지움. **보낼 때:** 세 칸 검사 → 틀리면 첫 칸에 커서, 끝 → 버튼 잠금 → PUT → 성공이면 비우고 안내, 400 이면 지금 칸, 422 면 새 칸, 그 밖은 폼 아래 → 잠금 풀기.

5. 왜 이렇게

선택	이유
가입 화면(16)과 같은 규칙 · 문구 · 시점	같은 비밀번호 규칙을 두 화면에서 다르게 보여 주지 않는다 (조건 미달은 벗어날 때, 한글은 바로, 확인은 다 친 뒤)
규칙만 공용 파일로 (화면 동작 코드는 각자)	가입의 16 은 검토가 끝난 코드다. 화면 동작까지 묶으려면 16 을 다시 짜야 해서, 템플릿이 권한 만큼(규칙)만 옮겼다. 비슷한 화면 코드가 두 곳에 있는 것은 나중에 정리할 수 있다
지금 비밀번호가 틀리면 그 칸을 비움	로그인 모달(P1-07 13)과 같음 — 다시 치게

선택	이유
성공하면 세 칸을 비움	브라우저에 비밀번호가 남아 있지 않게. 조건 목록도 ○ 로
다시 로그인시키지 않음	서버도 새 토큰을 주지 않는다 (47 §5) — 지금 로그인은 그대로
"Password changed." 5초 (닉네임 Saved 는 3초)	다른 칸이 비어 버려 변화가 커서, 무엇이 일어났는지 읽을 시간을 조금 더

6. 시험 (실제 userInfo.html · 실제 서버 · 12개)

시험	확인한 것
`Studio1`	대 · 소 · 숫자 ✓, 입력 중 문구 없음 → 벗어나면 "Check the password requirements."
한글	바로 "Use only English ..."
확인 칸	같은 길이인데 다름 → 문구 / 같아지면 지움, 조건 5개 ✓
빈 칸으로 보냄	세 칸 문구, 커서 = 지금 비밀번호, 요청 0번
지금 비밀번호 틀림	보내는 동안 버튼 잠금 → "Current password is incorrect.", 그 칸 비움 · 커서
새 = 지금	새 칸 "Use a different password.", 커서
바꾸기	세 칸 · 조건 · 문구 초기화, "Password changed." → **실제 로그인 API** 로 새 비밀번호 200, 옛 비밀번호 401
되돌리기	원래 비밀번호로 로그인 200

12/12 통과. 22 · 23 · 24 와 가입 화면(P1-07 16 · 17) 시험도 다시 돌려 모두 통과 (가입 시험이 만든 계정은 지움).